

Vasic-Digital-Reusable-Module-Suite

Vasic Digital Reusable Module Suite (`digital.vasic.*`)

Tagline: Build once, reuse everywhere — a fleet of small, decoupled, independently-tested Go and KMP modules.

Summary: A large family of generic, reusable modules published under the `digital.vasic.*` (Go) and Kotlin Multiplatform namespaces. Every module is standalone, independently tested and versioned, and consumed as an equal-codebase submodule by larger products (Catalogizer, HelixAgent, and the wider fleet). This brief consolidates the many small utilities that would be noise as individual pages.

Short description (~40 words): A curated suite of decoupled `digital.vasic.*` modules — infrastructure primitives (auth, cache, database, config, observability), AI/agent building blocks (RAG, VectorDB, Embeddings, MCP, Agentic, Planning), and defensive-LLM guardrails (RedTeam, Normalize) — plus a Kotlin Multiplatform mirror set. Each is generic, tested, and reusable.

Long description (150-250 words): The vasic-digital org is built on a “constitution + many decoupled reusable submodules” philosophy: rather than monoliths, generic functionality is extracted into dozens of small modules, each with its own tests, docs, and repository, and each strictly decoupled so no consumer’s specifics leak in. This brief groups them because individually they are library-scale, but together they are a significant engineering asset and a strong “we don’t reinvent the wheel” story.

The suite spans three clusters. **Infrastructure primitives** (Go) provide the plumbing every service needs: auth (JWT/bcrypt), cache (Redis/TTL), database (migrations, dual SQLite/PostgreSQL), config, middleware, observability (Prometheus/OpenTelemetry), ratelimiter, security, storage (S3/MinIO), streaming (WebSocket hub), eventbus, filesystem (multi-protocol), discovery/mdns, http3,

recovery, concurrency, lazy, and more. **AI/agent building blocks** (Go) provide the substrate for AI systems: rag, vectordb, embeddings, memory, conversation (infinite-context compression, event sourcing), mcp (Model Context Protocol), toolschema, skillregistry, agentic (graph-based workflow orchestration), planning (HiPlan/MCTS/Tree-of-Thoughts), benchmark (SWE-bench/HumanEval/MMLU), llmops, selfimprove (reward modeling/RLHF), and toon (Token-Oriented Object Notation). **Defensive-LLM guardrails** provide adversarial-robustness tooling: RedTeam (YAML-driven adversarial fixtures), Normalize (adversarial-input canonicalisation). A parallel **Kotlin Multiplatform** set mirrors core modules (Auth-KMP, Database-KMP, Security-KMP, UI-Components-KMP, etc.) for cross-platform apps.

Why we built it: Shipping many products (Catalogizer, HelixAgent, Herald, and more) from scratch each time is wasteful and inconsistent. Extracting every generic concern into a decoupled, tested module means fixes and improvements propagate across the whole fleet, and each new product assembles from proven parts.

Why it's a game-changer: It is a private “standard library” for building AI-centric backends: infrastructure, AI primitives, and guardrails all as drop-in, independently-tested modules — enabling small teams to ship product-grade systems fast without accumulating duplication.

What's innovative: - Fleet-wide decoupling discipline (CONST-051): submodules treated as equal codebases, never carrying consumer specifics. - A dedicated AI-primitive layer (RAG, VectorDB, Embeddings, MCP, ToolSchema, Agentic, Planning, LLMOps) as reusable modules. - A defensive-LLM guardrail cluster (RedTeam, Normalize) for adversarial robustness. - Parallel Go + Kotlin Multiplatform module sets sharing the same conventions.

Biggest technical challenges + how solved: - *Avoiding coupling rot across dozens of modules:* solved with the constitution's decoupling contract and runtime injection of consumer specifics. - *Keeping many modules consistent and tested:* solved with a shared convention (per-module tests/docs/Challenges) and the HelixConstitution governance backbone. - *Cross-platform reach:* solved with a Kotlin Multiplatform mirror of core modules.

Tech stack (why + how): - **Go** — the majority of modules (digital.vasic.*). - **Kotlin Multiplatform** — cross-platform mirror modules (Auth/Database/Security/UI/Concurrency/RateLimiter-KMP). - **Redis / PostgreSQL / SQLite** — cache, database, storage primitives. - **Prometheus / OpenTelemetry** — observability module. - **WebSocket / HTTP/3 (quic-go) / mDNS**

— networking modules. - **Vector DB / embeddings / RAG / MCP**
— AI-primitive modules. - **YAML** — RedTeam adversarial fixtures and config.

Public links: - GitHub org: <https://github.com/vasic-digital> (modules include auth, cache, database, config, observability, security, storage, streaming, eventbus, ratelimiter, middleware, RAG, VectorDB, Embeddings, Memory, conversation, MCP_Module, ToolSchema, SkillRegistry, Agentic, Planning, Benchmark, LLM0ps, SelfImprove, T00N, RedTeam, Normalize, http3, mdns, and the *-KMP set) — all public. - **UNVERIFIED / WIP:** Several org repos are self-marked “SCAFFOLD / WIP” (e.g. PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroborous, Claritas, LeakHub, GandalfSolutions) — present them as early-stage/scaffold, not shipped.

Suggested diagrams/illustrations (OpenDesign): 1. Three-cluster module map (Infrastructure / AI primitives / Guardrails) with product apps on top. 2. “Standard library” grid of modules, colored by maturity (stable vs scaffold). 3. Go ↔ KMP mirror pairs. 4. A product (e.g. Catalogizer or HelixAgent) exploded into the modules it consumes.

Site relevance: vasic.digital (engineering-depth / reusability / AI-infrastructure). A credibility exhibit rather than a single product.

Priority tier: vasic-util-secondary

Source provenance: _analysis/github-vasic-digital.md (full repo list with per-module descriptions and SCAFFOLD/WIP flags); / Volumes/T7/Projects/catalogizer/README.md (21-module digital.vasic.* table + KMP/TS modules); gh repo view vasic-digital/LLMProvider (module conventions). WIP items flagged per constitution §11.4.6.